CS-471: Machine Learning

Week 2 — Gradient Descent Step Size, Model Evaluation & Binary Classification

Instructor: Dr. Hashir Kiani Sources: Lectures 2-1 & 2-2

Comprehensive Study Notes

Week 2

Contents

1 Choosing the Step Size (Learning Rate) in Gradient Descent **2**
- 1.1 Gradient Descent Recap . . . 2
- 1.2 Strategies for Choosing α . . . 2
  - 1.2.1 Constant Step Size . . . 2
  - 1.2.2 Decreasing Step Size . . . 2

2 Linear Regression — Worked Example **3**
- 2.1 Problem Setup . . . 3

3 Training, Validation, and Testing Sets **4**
- 3.1 Why Split the Data? . . . 4
- 3.2 The Three Splits . . . 4
- 3.3 Cross-Validation (Bonus Concept) . . . 5

4 Regression Model Performance Evaluation **5**
- 4.1 Mean Squared Error (MSE) . . . 5
- 4.2 Root Mean Squared Error (RMSE) . . . 6
- 4.3 Mean Absolute Error (MAE) . . . 6
- 4.4 R^2 (Coefficient of Determination) . . . 6

5 Binary Linear Classification **7**
- 5.1 What is Binary Classification? . . . 7
- 5.2 The Three Pillars (Applied to Classification) . . . 7
- 5.3 Hypothesis Class: The Linear Classifier . . . 8
  - 5.3.1 Concrete Example . . . 8

6 Score, Margin, and the Zero-One Loss **8**
- 6.1 Score . . . 8
- 6.2 Margin . . . 9
- 6.3 Zero-One Loss . . . 9

7 Hinge Loss **9**
7.1 Motivation . . . 9
7.2 Definition . . . 10
7.3 Graph Interpretation . . . 10
7.4 Gradient of Hinge Loss . . . 10
7.5 Gradient Descent with Hinge Loss . . . 11

8 Logistic Regression **11**
8.1 Motivation . . . 11
8.2 Hypothesis Class: The Sigmoid Function . . . 11
8.3 Probabilistic Interpretation . . . 12
8.4 Sigmoid Graph Interpretation . . . 12

9 Worked Examples **13**
9.1 Example 1 — Evaluating a Regression Model . . . 13
9.2 Example 2 — Hinge Loss Full Computation . . . 13
9.3 Example 3 — Sigmoid Output . . . 14

10 Practice Questions **14**

11 Key Takeaways — Week 2 **15**

1 Choosing the Step Size (Learning Rate) in Gradient Descent

1.1 Gradient Descent Recap

The weight update rule for gradient descent is:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha \, \nabla_{\mathbf{w}} \, \text{TrainLoss}(\mathbf{w})$$

where α (also written η) is the **step size** (learning rate).

1.2 Strategies for Choosing α

Choosing α is one of the most important decisions when training a model. The lecture presents two main strategies:

1.2.1 Constant Step Size

$$\alpha = \text{constant} \quad (\text{e.g. } 0.01,\ 0.001)$$

- Simple to implement.
- Works well when the loss surface is smooth and not too steep.
- **Risk:** Too large $\Rightarrow$ overshoots the minimum (diverges); too small $\Rightarrow$ convergence is very slow.

1.2.2 Decreasing Step Size

$$\alpha_t = \frac{\alpha_0}{\sqrt{t}}$$

where t is the *number of updates made so far* and α_0 is the *initial step size*.

- As training progresses, steps become smaller.
- Takes large exploratory steps early; fine-tunes near the end.
- Guarantees convergence under standard conditions.
- Common default: start with $\alpha_0 = 1$ and let $\alpha_t = 1/\sqrt{t}$.

Key Idea

Intuition for decreasing step size:
Imagine hiking toward a valley. When you are far from the bottom, you take big strides. As you approach the bottom, you slow down to avoid stepping over it. The formula $1/\sqrt{t}$ is a mathematically principled way to emulate this behaviour.

Property	Constant α	Decreasing $\alpha_t = \alpha_0/\sqrt{t}$
Convergence guarantee	Only near minimum	Yes (under mild conditions)
Ease of tuning	Medium	Easier (only tune α_0)
Speed early on	Depends on choice	Fast (large initial steps)
Speed late on	Depends on choice	Slow (fine adjustment)

Real-Life Analogy: A student studying for exams. Early in the semester they read chapters broadly (large steps). The night before the exam they review specific details carefully (small, precise steps).

2 Linear Regression — Worked Example

2.1 Problem Setup

The lecture provides the following training dataset:

$\mathbf{x}$ (feature vector)	y (label)
$[1,\ 0]$	2
$[1,\ 0]$	4
$[0,\ 1]$	1

Feature extractor: $\boldsymbol{\phi}(\mathbf{x}) = [1,\ x_1,\ x_2]$ (prepend bias term 1).

So the three training examples become:

$$\boldsymbol{\phi}^{(1)} = [1,\ 1,\ 0], \quad \boldsymbol{\phi}^{(2)} = [1,\ 1,\ 0], \quad \boldsymbol{\phi}^{(3)} = [1,\ 0,\ 1]$$

with labels $y^{(1)} = 2,\ y^{(2)} = 4,\ y^{(3)} = 1$.

Example

Goal: Starting from $\mathbf{w} = [0,\ 0,\ 0]$, perform one step of batch gradient descent with $\alpha = 0.5$.

Step 1 — Compute predictions ($h = \mathbf{w}^\top \boldsymbol{\phi}$):

$$h^{(1)} = 0, \quad h^{(2)} = 0, \quad h^{(3)} = 0$$

Step 2 — Residuals ($r^{(i)} = h^{(i)} - y^{(i)}$):

$$r^{(1)} = -2, \quad r^{(2)} = -4, \quad r^{(3)} = -1$$

Step 3 — Gradient $\nabla_{\mathbf{w}}\,\text{TrainLoss} = \frac{1}{N}\sum r^{(i)}\boldsymbol{\phi}^{(i)}$:

$$\nabla_{w_0} = \frac{1}{3}[(-2)(1) + (-4)(1) + (-1)(1)] = \frac{-7}{3} \approx -2.33$$
$$\nabla_{w_1} = \frac{1}{3}[(-2)(1) + (-4)(1) + (-1)(0)] = \frac{-6}{3} = -2$$
$$\nabla_{w_2} = \frac{1}{3}[(-2)(0) + (-4)(0) + (-1)(1)] = \frac{-1}{3} \approx -0.33$$

Step 4 — Update weights ($\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla$):

$$w_0 \leftarrow 0 - 0.5 \times (-2.33) = +1.17$$
$$w_1 \leftarrow 0 - 0.5 \times (-2.00) = +1.00$$
$$w_2 \leftarrow 0 - 0.5 \times (-0.33) = +0.17$$

Interpretation: All weights increased, which makes sense — all predictions were too low, so the model needed to predict higher values. After many such iterations, the weights will converge to values that best fit the training data.
Note that examples 1 and 2 have the *same* features $[1, 1, 0]$ but different labels (2 and 4). A linear model cannot perfectly fit contradictory data; it will settle on a prediction somewhere between 2 and 4 for that feature vector.

Alternative Method / Trick

Normal Equation (closed-form solution):
For linear regression, there is an exact solution without iterating:

$$\mathbf{w}^* = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y}$$

where Φ is the **design matrix** (rows = feature vectors). Gradient descent is preferred for large N or n because matrix inversion costs $O(n^3)$; GD is more scalable.

3 Training, Validation, and Testing Sets

3.1 Why Split the Data?

A machine learning model is trained to make predictions on data it has *never seen before*. If we evaluate its performance only on the data used to train it, we get an overly optimistic estimate — the model may have simply memorised the training data (**overfitting**).

3.2 The Three Splits

Split	Role	Decisions Made Using It
Training Set	Used to **learn** the model parameters ($\mathbf{w}$).	Gradient descent runs on this data.
Validation Set	Used to **evaluate** the model during development and tune hyperparameters.	Choose learning rate α, model complexity, number of features.
Testing Set	Used **once** at the end to measure the final model's generalisation performance.	Reports the accuracy/error you would publish or ship.

Common Mistake / Pitfall

Critical Rule: Never use the test set during training or hyperparameter tuning. Doing so leaks information from the test set into model decisions, making the test evaluation optimistic and unreliable.

Typical Splits:

- **Small dataset** ($N < 10{,}000$): 60% train / 20% val / 20% test.
- **Large dataset** ($N > 10^6$): 98% train / 1% val / 1% test is acceptable, since even 1% of 10 million is 100 k examples.

Real-Life Analogy: Think of studying for an exam with three sets of problems:

- **Training set** $\Rightarrow$ the practice problems you study from.
- **Validation set** $\Rightarrow$ mock exams you take to see how ready you are and adjust your study strategy.
- **Test set** $\Rightarrow$ the final exam — you only sit it once.

3.3 Cross-Validation (Bonus Concept)

When data is scarce, **k-fold cross-validation** is used:

1. Split data into k equal folds.
2. Train on $k-1$ folds, validate on the remaining fold.
3. Repeat k times, rotating the validation fold.
4. Average the k validation scores for a stable estimate.

4 Regression Model Performance Evaluation

After training a regression model, we need metrics to measure how well it performs. The lecture covers four standard metrics.

4.1 Mean Squared Error (MSE)

$$\boxed{\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(h(\mathbf{x}^{(i)}) - y^{(i)}\right)^2}$$

- Average of the squared residuals.
- Penalises large errors heavily.
- Units: (unit of y)^2 — not directly interpretable as a dollar or degree error.

4.2 Root Mean Squared Error (RMSE)

$$\boxed{\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(h(\mathbf{x}^{(i)}) - y^{(i)}\right)^2}}$$

- Square root of MSE.
- **Same units as** y — directly interpretable (e.g. "the model is off by \$15,000 on average").
- The most commonly reported metric for regression.

4.3 Mean Absolute Error (MAE)

$$\boxed{\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}\left|h(\mathbf{x}^{(i)}) - y^{(i)}\right|}$$

- Average of the absolute (not squared) residuals.
- Less sensitive to *outliers* than MSE/RMSE.
- Useful when outliers should not disproportionately influence the metric.

4.4 R^2 (Coefficient of Determination)

$$\boxed{R^2 = 1 - \frac{SS_{\text{residual}}}{SS_{\text{total}}}}$$

where:

$$SS_{\text{residual}} = \sum_{i=1}^{N}\left(y^{(i)} - h(\mathbf{x}^{(i)})\right)^2 \quad \text{(sum of squared prediction errors)}$$

$$SS_{\text{total}} = \sum_{i=1}^{N}\left(y^{(i)} - \bar{y}\right)^2 \quad \text{(total variance in } y\text{; } \bar{y} = \text{mean of labels)}$$

Interpretation:

- $R^2 = 1$: Perfect predictions. Model explains 100% of variance.
- $R^2 = 0$: Model does no better than simply predicting the mean $\bar{y}$ for all inputs.
- $R^2 < 0$: Model is *worse* than the mean baseline (strongly overfitted or wrong model).
- Range: $(-\infty, 1]$; higher is better.

Real-Life Analogy: R^2 measures how much of the "uncertainty" in house prices your model explains. An $R^2 = 0.85$ means your model accounts for 85% of why one house costs more than another. The remaining 15% is due to factors not in your model.

Metric	Formula	Units	Outlier Sensitivity	Higher = Better?
MSE	$\frac{1}{N}\sum r_i^2$	y^2	High	No (lower)
RMSE	$\sqrt{\text{MSE}}$	y	High	No (lower)
MAE	$\frac{1}{N}\sum \lvert r_i \rvert$	y	Low	No (lower)
R^2	$1 - SS_{\text{res}}/SS_{\text{tot}}$	None	Medium	Yes

Example

Mini Example — Compute all four metrics:
Predictions $\hat{y} = [3,\ 5,\ 2]$, True labels $y = [2,\ 4,\ 4]$, $\bar{y} = 10/3 \approx 3.33$.
Residuals: $r = [1,\ 1,\ -2]$

$$\text{MSE} = \frac{1^2 + 1^2 + (-2)^2}{3} = \frac{6}{3} = 2$$

$$\text{RMSE} = \sqrt{2} \approx 1.41$$

$$\text{MAE} = \frac{|1| + |1| + |-2|}{3} = \frac{4}{3} \approx 1.33$$

$$SS_{\text{res}} = 1 + 1 + 4 = 6, \quad SS_{\text{tot}} = (2 - 3.33)^2 + (4 - 3.33)^2 + (4 - 3.33)^2 = 1.78 + 0.44 + 0.44 = 2.6$$

$$R^2 = 1 - \frac{6}{2.67} \approx 1 - 2.25 = -1.25$$

The negative R^2 means these predictions are *worse* than just predicting the mean — the model needs more training.

5 Binary Linear Classification

5.1 What is Binary Classification?

Binary classification is supervised learning where the output label $y \in \{+1,\ -1\}$ (or equivalently $\{1,\ 0\}$).

Real-World Examples:

- **Spam detection:** Is an email spam $(+1)$ or not (-1)?
- **Fraud detection:** Is a credit card transaction fraudulent $(+1)$ or legitimate (-1)?
- **Medical diagnosis:** Does an X-ray show a tumour $(+1)$ or not (-1)?

Extension — Multiclass Classification: When there are more than two possible labels (e.g. dog, cat, bird), we have *multiclass* classification. Binary classification is the building block.

5.2 The Three Pillars (Applied to Classification)

Just like linear regression, binary classification needs:

1. **Hypothesis Class** — the family of classifiers we consider.
2. **Loss Function** — measures how wrong a prediction is.
3. **Optimisation Algorithm** — finds the best classifier.

5.3 Hypothesis Class: The Linear Classifier

$$\boxed{h(\mathbf{x}) = \text{sign}\big(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})\big)}$$

where:

$$\text{sign}(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$$

Variable Definitions:

- $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$ — the **score** (real number). Tells us how strongly the model leans toward $+1$.
- $\text{sign}(\cdot)$ — converts the continuous score into a binary prediction.
- The **decision boundary** is where score $= 0$, i.e. $\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) = 0$ — a *hyperplane* in feature space.

Geometric Intuition: A linear classifier draws a straight line (in 2D) or a hyperplane (in higher dimensions) that separates the two classes. Points on one side are classified $+1$; on the other side -1.

5.3.1 Concrete Example

The lecture gives: $\mathbf{w} = [0,\ -0.6,\ 0.6]$, $\boldsymbol{\phi}(\mathbf{x}) = [1,\ x_1,\ x_2]$.

Score $= 0(1) + (-0.6)x_1 + (0.6)x_2 = 0.6(x_2 - x_1)$.

- If $x_2 > x_1$: score $> 0 \Rightarrow h = +1$.
- If $x_2 < x_1$: score $< 0 \Rightarrow h = -1$.
- The decision boundary is the line $x_2 = x_1$.

Real-Life Analogy: Imagine classifying emails. Each email has a "spam score"; above zero it is marked spam, below zero it goes to the inbox. The linear classifier is the rule that computes that score from the email's features (word frequencies, sender, etc.).

6 Score, Margin, and the Zero-One Loss

6.1 Score

$$\text{Score} = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$$

The score is a real-valued number that reflects how confidently the model predicts $+1$:

- Large positive score $\Rightarrow$ very confident prediction of $+1$.

- Large negative score $\Rightarrow$ very confident prediction of -1.
- Score near zero $\Rightarrow$ uncertain prediction.

6.2 Margin

$$\text{Margin} = y\,(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}))$$

The margin combines the true label $y \in \{+1, -1\}$ with the score:

- **Positive margin** $\Rightarrow$ correct prediction (score and label have the same sign).
- **Negative margin** $\Rightarrow$ incorrect prediction (score and label have opposite signs).
- **Larger margin** $\Rightarrow$ more confidently correct.

Key Idea

Key insight: The margin is the unified measure of *correctness with confidence*. A large positive margin means "correctly classified, and I am sure of it." A large negative margin means "incorrectly classified, and I was very wrong about it."

6.3 Zero-One Loss

$$\boxed{\text{Loss}_{\text{0-1}}(\mathbf{x},\, y,\, \mathbf{w}) = \mathbf{1}[h(\mathbf{x}) \neq y]}$$

This equals 1 if the prediction is wrong, 0 if it is correct.

- Directly measures classification *accuracy* (number of errors).
- Intuitive and easy to interpret.

Common Mistake / Pitfall

Critical Problem with Zero-One Loss and Gradient Descent:
The zero-one loss is a *step function* — it is completely flat everywhere except at the decision boundary. Therefore its gradient is **zero almost everywhere**. Gradient descent cannot move in any direction because the gradient gives no information about which way to update $\mathbf{w}$. This makes zero-one loss **non-differentiable** and unsuitable for gradient-based optimisation.

Real-Life Analogy of the problem: Imagine you are blindfolded on a flat plain with a sudden cliff edge somewhere. You want to walk away from the cliff, but the ground is perfectly flat so you cannot feel any slope — no gradient. You have no information about which way to move. That is exactly the situation with the zero-one loss.

7 Hinge Loss

7.1 Motivation

We need a loss that:

1. Is **differentiable** (so gradient descent works).
2. Is a **good proxy** for the zero-one loss.
3. Penalises not just misclassifications, but also predictions that are "not confident enough".

7.2 Definition

$$\boxed{\text{Loss}_{\text{hinge}}(\mathbf{x},\, y,\, \mathbf{w}) = \max\{1 - y\,(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})),\ 0\}}$$

equivalently written as $\max\{1 - \text{margin},\ 0\}$.

Breakdown:

- If margin ≥ 1: loss $= 0$. The prediction is correct *and* confident enough.
- If margin $\in [0, 1)$: loss $= 1 - \text{margin} > 0$. Prediction is technically correct (positive margin) but not confident enough — the model is "rewarded" for being more certain.
- If margin < 0: loss $= 1 - \text{margin} > 1$. Prediction is wrong.

Key Idea

Hinge loss requires a margin of at least 1 to contribute zero loss. This enforces a *safety margin* around the decision boundary — the model is trained to be confidently correct, not just barely correct. This is the core idea behind **Support Vector Machines (SVMs)**, which are covered in a later week.

7.3 Graph Interpretation

The hinge loss (green line on the lecture slide) is a piecewise linear function of the margin:

Margin $= y \cdot \text{score}$	Hinge Loss
< 0	$1 - \text{margin} > 1$ (large loss)
$= 0$	1 (prediction is on the boundary)
$= 1$	0 (loss kicks to zero — "safe margin" reached)
> 1	0 (confidently correct — no loss)

Shape: Decreasing linearly until margin $= 1$, then flat at 0. This "hinge" shape gives the loss its name.

7.4 Gradient of Hinge Loss

$$\nabla_{\mathbf{w}} \text{Loss}_{\text{hinge}}(\mathbf{x},\, y,\, \mathbf{w}) = \begin{cases} -y\,\boldsymbol{\phi}(\mathbf{x}) & \text{if } y\,(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})) < 1 \quad \text{(margin too small)} \\ \mathbf{0} & \text{otherwise} \quad \text{(margin} \geq 1\text{, no update needed)} \end{cases}$$

Intuition:

- If the margin is already ≥ 1, the hinge loss is 0 and we do nothing (gradient is zero — stop perturbing a good predictor).

- If the margin is < 1 (either too uncertain or outright wrong), we push the weight vector in the direction $y\,\boldsymbol{\phi}(\mathbf{x})$, which increases the margin for this example.

7.5 Gradient Descent with Hinge Loss

Plugging the hinge gradient into the GD update rule:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \alpha\,\nabla_{\mathbf{w}}\,\text{TrainLoss}(\mathbf{w})$$

Expanding the training loss gradient (average over all N examples):

$$\boxed{\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \frac{\alpha}{N}\sum_{i=1}^{N}\mathbf{1}\big[y^{(i)}(\mathbf{w}^\top\boldsymbol{\phi}(\mathbf{x}^{(i)})) < 1\big]\cdot y^{(i)}\,\boldsymbol{\phi}(\mathbf{x}^{(i)})}$$

(The negative from $-\alpha\nabla$ and $-y\phi$ cancel to give $+$.)

Step-by-step reading: For each training example, if its margin is less than 1, add $\frac{\alpha}{N}\,y^{(i)}\boldsymbol{\phi}(\mathbf{x}^{(i)})$ to $\mathbf{w}$. Otherwise, skip it.

Example

One-step SGD Example with Hinge Loss:
Setup: $\mathbf{w} = [0,\ 0.5,\ 1]$, $\boldsymbol{\phi}(\mathbf{x}) = [1,\ 2,\ -1]$, $y = +1$, $\alpha = 0.1$.
Score: $\mathbf{w}^\top\boldsymbol{\phi} = 0(1) + 0.5(2) + 1(-1) = 1 - 1 = 0$
Margin: $y \times 0 = 0 < 1 \Rightarrow$ update needed.
Gradient contribution: $-y\boldsymbol{\phi} = -(+1)[1, 2, -1] = [-1,\ -2,\ 1]$
Update: $\mathbf{w} \leftarrow [0, 0.5, 1] - 0.1 \times [-1, -2, 1] = [0.1,\ 0.7,\ 0.9]$
Verify: New score $= 0.1(1) + 0.7(2) + 0.9(-1) = 0.1 + 1.4 - 0.9 = 0.6$. Margin $= 0.6 > 0$ — the model is now predicting the right direction, but we need one more update to breach margin 1.

8 Logistic Regression

8.1 Motivation

The linear classifier with a sign function outputs $+1$ or -1 with no sense of *probability*. Logistic regression adds a probabilistic interpretation.

8.2 Hypothesis Class: The Sigmoid Function

$$\boxed{h(\mathbf{x}) = g\big(\mathbf{w}^\top\boldsymbol{\phi}(\mathbf{x})\big)\,, \quad \text{where} \quad g(z) = \frac{1}{1+e^{-z}}}$$

The function $g(z)$ is called the **sigmoid** (or **logistic**) function.

Properties of the sigmoid:

- Domain: $z \in (-\infty, +\infty)$.

- Range: $(0, 1)$ — output is always between 0 and 1.
- $g(0) = 0.5$ — uncertain input maps to 50% probability.
- As $z \to +\infty$: $g(z) \to 1$.
- As $z \to -\infty$: $g(z) \to 0$.
- Smooth, differentiable everywhere — gradient descent works.

8.3 Probabilistic Interpretation

$$h(\mathbf{x}) = g(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})) \approx P(y = +1 \mid \mathbf{x};\ \mathbf{w})$$

The output is interpreted as the **probability of the positive class**:

- $g(z) > 0.5 \Rightarrow z > 0 \Rightarrow$ predict $+1$.
- $g(z) \leq 0.5 \Rightarrow z \leq 0 \Rightarrow$ predict -1.

Real-Life Analogy: A doctor looking at patient symptoms gives not just "has disease / doesn't have disease" but a probability: "There is a 72% chance this patient has disease X." Logistic regression gives you that probability.

The relationship between Logistic Regression and Linear Classification:

Property	Linear Classifier (sign)	Logistic Regression (sigmoid)
Output	Hard $\{-1, +1\}$	Soft $(0, 1)$ probability
Decision boundary	$\mathbf{w}^\top \boldsymbol{\phi} = 0$	Same
Differentiable?	No (at boundary)	Yes (everywhere)
Probabilistic?	No	Yes

8.4 Sigmoid Graph Interpretation

The sigmoid curve:

- **X-axis** (z): the score / linear combination of features.
- **Y-axis** ($g(z)$): the predicted probability of class $+1$.
- **Shape**: S-shaped ("sigmoidal").
- **Trend**: montonically increasing — larger score always gives higher probability; smaller score always gives lower probability.
- **Asymptotes**: flattens at 0 and at 1, never actually reaching them.
- **Steepest point**: $z = 0$ (the decision boundary).

Key Idea

Why the sigmoid is so useful: It "squashes" any real number into a valid probability. You can pour any score from $-\infty$ to $+\infty$ into the sigmoid and out comes a number between 0 and 1 — ready to be interpreted as a probability.

9 Worked Examples

9.1 Example 1 — Evaluating a Regression Model

Example

Three houses, true prices $y = [200, 350, 500]$ (thousands), model predictions $\hat{y} = [210, 340, 480]$.

Residuals: $[10,\ -10,\ -20]$

$$\begin{aligned}
\text{MSE} &= \frac{100 + 100 + 400}{3} = 200 \\
\text{RMSE} &= \sqrt{200} \approx 14.14 \text{ thousand} \\
\text{MAE} &= \frac{10 + 10 + 20}{3} \approx 13.33 \text{ thousand} \\
\bar{y} &= \frac{200 + 350 + 500}{3} = 350 \\
SS_{\text{tot}} &= (200 - 350)^2 + (350 - 350)^2 + (500 - 350)^2 = 22500 + 0 + 22500 = 45000 \\
SS_{\text{res}} &= 100 + 100 + 400 = 600 \\
R^2 &= 1 - \frac{600}{45000} \approx 1 - 0.013 = 0.987
\end{aligned}$$

Interpretation: RMSE = \$14,140 typical error. $R^2 = 0.987$ means the model explains 98.7% of variance — excellent fit.

9.2 Example 2 — Hinge Loss Full Computation

Example

Binary classification with two training examples:

i	$\boldsymbol{\phi}(\mathbf{x}^{(i)})$	$y^{(i)}$	score $= \mathbf{w}^\top \boldsymbol{\phi}$
1	$[1,\ 2]$	$+1$	$\mathbf{w}^\top[1, 2] = w_0 + 2w_1$
2	$[1,\ -1]$	-1	$\mathbf{w}^\top[1, -1] = w_0 - w_1$

With $\mathbf{w} = [1,\ 1]$:
Example 1: score $= 3$, margin $= (+1)(3) = 3 \geq 1 \Rightarrow$ hinge loss $= 0$.
Example 2: score $= 0$, margin $= (-1)(0) = 0 < 1 \Rightarrow$ hinge loss $= 1 - 0 = 1$.
Training loss $= (0 + 1)/2 = 0.5$.
Gradient for example 2 (margin < 1): $-y^{(2)}\boldsymbol{\phi}^{(2)} = -(-1)[1, -1] = [1,\ -1]$
One SGD step ($\alpha = 0.1$): $\mathbf{w} \leftarrow [1, 1] - 0.1[1, -1] = [0.9,\ 1.1]$

9.3 Example 3 — Sigmoid Output

Example

Compute $h(\mathbf{x})$ for logistic regression with $\mathbf{w} = [2, -3]$, $\boldsymbol{\phi}(\mathbf{x}) = [1,\ 1]$.
Score $= 2(1) + (-3)(1) = -1$
$g(-1) = \frac{1}{1+e^1} = \frac{1}{1+2.718} = \frac{1}{3.718} \approx 0.269$
Interpretation: 26.9% probability of class $+1$; since $0.269 < 0.5$, predict -1.

10 Practice Questions

Practice Questions

Topic: Step Size, Train/Val/Test

Q1 (Easy): What is the formula for a decreasing step size, and why is a decreasing step size preferable to a constant step size when training?
Answer: $\alpha_t = \alpha_0/\sqrt{t}$. A decreasing schedule takes large exploratory steps early in training (when far from the optimum) and smaller, finer steps later (when close to the optimum), reducing the risk of overshooting while still converging efficiently.

Q2 (Medium): A dataset has 50,000 examples. You split it 60/20/20. You notice the validation loss stops improving after epoch 10 but the training loss keeps decreasing. What is happening and what should you do?
Answer: The model is **overfitting** — it is memorising the training data rather than learning generalisable patterns. Actions: stop training at epoch 10 (early stopping), reduce model complexity, add regularisation (L2/L1), or collect more data.

Q3 (Exam-level): Explain why the test set must be used only once and what happens if you tune hyperparameters based on test set performance.
Answer: Repeatedly evaluating on the test set and adjusting the model in response is equivalent to *training* on the test set. Your reported test accuracy becomes optimistic because you have inadvertently fitted to the test distribution. The model will then underperform on truly new data — defeating the entire purpose of holding out a test set.

Topic: Classification & Loss Functions

Q4 (Easy): Given $\mathbf{w} = [1,\ 2,\ -1]$ and $\boldsymbol{\phi}(\mathbf{x}) = [1,\ 1,\ 2]$, what is the prediction of the linear classifier?
Score $= 1(1) + 2(1) + (-1)(2) = 1 + 2 - 2 = 1$. $\text{sign}(1) = +1 \Rightarrow$ predict $+1$.

Q5 (Medium): Compute the hinge loss for $y = -1$, margin $= -0.5$. Then write the gradient and perform one SGD step with $\alpha = 0.2$, $\boldsymbol{\phi}(\mathbf{x}) = [1,\ 3]$.
Hinge loss $= \max\{1 - (-0.5), 0\} = \max\{1.5, 0\} = 1.5$.
Gradient: margin $= -0.5 < 1 \Rightarrow -y\boldsymbol{\phi} = -(-1)[1, 3] = [1, 3]$.
$\mathbf{w} \leftarrow \mathbf{w} - 0.2[1, 3] = \mathbf{w} + [-0.2, -0.6]$.

Q6 (Exam-level): Compare zero-one loss and hinge loss. Why can gradient descent not be used with zero-one loss? How does hinge loss solve this problem? What geometric property does hinge loss optimise for?
Answer:
Zero-one loss counts raw errors (0 or 1) – it is a step function with gradient zero almost everywhere. GD cannot use a zero gradient to determine which direction to update $\mathbf{w}$.
Hinge loss is a differentiable (piecewise linear) proxy for zero-one loss. It has a non-zero gradient whenever the margin < 1, giving GD a clear update direction.
Geometrically, hinge loss not only corrects misclassifications but also enforces a *margin of at least 1* around the decision boundary, yielding a robust classifier (the foundation of SVMs).

Topic: Logistic Regression

Q7 (Easy): What is $g(0)$? What does it mean in terms of prediction?
$g(0) = 1/(1 + e^0) = 1/2 = 0.5$. The model is completely uncertain between $+1$ and -1, sitting exactly on the decision boundary.

Q8 (Medium): For $\mathbf{w} = [0,\ 1]$ and $\boldsymbol{\phi}(\mathbf{x}) = [1,\ 3]$, compute the probability of class $+1$.
Score $= 0 + 3 = 3$. $g(3) = 1/(1 + e^{-3}) = 1/(1 + 0.0498) \approx 0.953$. The model is 95.3% confident this is class $+1$.

11 Key Takeaways — Week 2

Quick Summary

- **Step size α:** can be constant or decreasing ($\alpha_t = \alpha_0/\sqrt{t}$). Decreasing rates guarantee convergence; constant rates need careful tuning.
- **Data splits:** Training (learn $\mathbf{w}$) / Validation (tune hyperparameters) / Test (final evaluation, used once).
- **Regression metrics:** MSE, RMSE (same units as y, most interpretable), MAE (robust to outliers), R^2 (proportion of variance explained, 0–1 ideal).
- **Linear classifier:** $h(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \boldsymbol{\phi})$. Decision boundary is hyperplane; margin $= y \cdot \text{score}$.
- **Zero-one loss:** intuitive but gradient $= 0$ (non-differentiable) $\Rightarrow$ GD fails.
- **Hinge loss:** $\max\{1 - y(\mathbf{w}^\top \boldsymbol{\phi}),\ 0\}$. Differentiable proxy; requires margin ≥ 1; gradient $= -y\boldsymbol{\phi}$ if margin < 1, else 0.
- **Logistic regression:** $h = g(\mathbf{w}^\top \boldsymbol{\phi})$, $g(z) = 1/(1 + e^{-z})$. Outputs probability; same linear decision boundary; fully differentiable.
- **Multiclass:** extension of binary classification to > 2 classes (covered in later weeks).